

# Project Presentation Guide

Pediatric RAG Pipeline — Build, Code, and Understanding

What to say · How to explain · What to demo · Expected questions

## Recommended Presentation Structure — 20 Minutes

| Step | Topic                                     | Time  | Goal                                  |
|------|-------------------------------------------|-------|---------------------------------------|
| 1    | Opening — The Problem                     | 2 min | Hook the audience — 'Why RAG at all?' |
| 2    | Visual Architecture                       | 3 min | Big picture before details            |
| 3    | Code walkthrough — build_index            | 4 min | Demonstrate technical understanding   |
| 4    | Code walkthrough — retrieval + generation | 3 min | End-to-end data flow                  |
| 5    | Evaluation results + ablation             | 4 min | Real numbers + analysis               |
| 6    | Lessons learned & improvements            | 2 min | Critical thinking depth               |
| 7    | Q&A                                       | 2 min | Intellectual flexibility              |

### Tip General advice

Always open with WHY before WHAT. Your instructor wants to see that you understood the problem the technology solves, not just that you wrote code that runs.

1

## Opening — The Problem Being Solved

Open with:

*Say: My project builds a RAG system — Retrieval-Augmented Generation — that lets you ask clinical questions about pediatric medical textbooks and receive answers grounded in the source text, with an exact page-level citation.*

Then immediately explain: why isn't plain GPT/Claude enough?

*Say: A general language model doesn't know the specific cases in the AAP 2022 book — Simon, a child with febrile seizure, or a mother with rapid breathing. It will produce an answer that sounds correct but fabricates details. RAG forces every answer to come directly from retrieved text — a real line from the book — and cites the exact page.*

### Tip Key moment

Say this sentence: 'The simple test is to ask the same question to Claude without RAG — it produces a medically plausible answer that appears in no textbook.' This proves you understand when RAG is actually necessary.

## 2 Architecture — The Full Picture

Show the architecture diagram and walk through each stage in one line:

|                            |                                                                                                                                                           |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PDF Files</b>           | Two books — 1,096 pages after skipping image-only pages (fewer than 50 characters).                                                                       |
| <b>Load + Clean</b>        | PyMuPDF extracts text page-by-page. <code>clean_text()</code> removes PDF artifacts: page numbers, triple blank lines, lone bullet characters.            |
| <b>Chunking</b>            | Two parallel strategies: fixed-size (400 chars, 50-char overlap) and paragraph-aware (100–700 chars at natural boundaries). Total: 10,107 + 5,250 chunks. |
| <b>Embedding</b>           | all-MiniLM-L6-v2 — 384-dimensional, L2-normalised. 227 seconds on CPU for 10 K chunks. Normalisation turns cosine similarity into a dot product.          |
| <b>numpy index</b>         | .npy + .json files. No external database. Originally ChromaDB — replaced due to a Windows bug that prevented persistence across processes.                |
| <b>Search + Generation</b> | scores = embeddings @ qvec.T → np.argmaxpartition → top-5 chunks → prompt to claude-haiku-4-5 → answer + [Source: ...] citation.                          |

### Warning Common mistake

Don't say 'I used ChromaDB.' You did use ChromaDB but discovered a Windows bug and migrated to numpy. That's exactly what an instructor wants to hear — that you hit a real problem and thought through a solution.

## 3 Code Walkthrough — build\_index.py

Open `src/build_index.py` and walk through the 4 main functions:

```
load_all_documents()
```

Function in `utils.py`. PyMuPDF opens each PDF, extracts text page-by-page, runs `clean_text()`, builds a `section_map` from the TOC. Returns a list of `{doc_id, text, metadata}`.

```
chunk_fixed_size(docs, size=400, overlap=50)
```

Sliding window. `step = size - overlap = 350`. Each step appends `chunk[text[i : i+400]]`. Tail < 30 chars is discarded.

```
embed_chunks(model, chunks, batch=128)
```

`model.encode(texts, normalize_embeddings=True)`. `normalize=True` applies L2 norm so cosine == dot product.

Returns float32 array of shape (N, 384).

```
save_index(chunks, embeddings, name)
```

`np.save(f'{name}_embeddings.npy', embeddings)` + `json.dump` chunks to file. Load with `np.load` — zero external dependencies.

Say: When you show `embed_chunks`, say: 'Notice `normalize_embeddings=True` — that's why we can compute cosine similarity with a plain @ and don't need to divide by vector magnitudes. It's also what lets us store everything in a plain .npy file instead of a specialised database.'

Code to know by heart:

```
vectors = model.encode(texts, batch_size=128, normalize_embeddings=True)
np.save('fixed_embeddings.npy', vectors.astype(np.float32))
```

## 4

### Code Walkthrough — retrieval.py + rag\_system.py

Open src/retrieval.py — show the retrieve() function:

```
qvec = model.encode([query], normalize_embeddings=True).astype(np.float32) scores = (embeddings @
qvec.T).flatten() # (N,) cosine similarity top_idx = np.argpartition(scores, -k)[-k:] # O(N), no full
sort top_idx = top_idx[np.argsort(scores[top_idx])[:-1]]
```

Say: This code performs a cosine search over 10,107 vectors in under a hundredth of a second. `np.argpartition` is  $O(N)$  — it doesn't sort everything, it just finds the top- $k$ . We then sort only those  $k$  elements — much faster than `argsort` over the full array.

Open src/rag\_system.py — show the answer() function:

```
chunks = retrieve(query, k=5, strategy='fixed') context = '\n'.join(c['text'] for c in chunks) prompt =
f'Context:\n{context}\n\nQuestion: {query}' response =
client.messages.create(model='claude-haiku-4-5-20251001', max_tokens=512, messages=[...])
```

Say: Notice the context is built from chunks sorted by descending score — the most relevant chunk is first. `max_tokens=512` is enough for a clinical answer with citations and prevents long paragraph overflows from the context window.

## 5

### Evaluation Results and Ablation Study

Show the tables from the report and explain every number:

| Metric              | Value             | How to explain it aloud                                                                                 |
|---------------------|-------------------|---------------------------------------------------------------------------------------------------------|
| Hit@5 = 0.827       | 43 / 52 questions | 'In almost 83% of questions, the text containing the answer appears in the top-5 chunks returned by     |
| Precision@5 = 0.881 | 4.4 out of 5      | 'Nearly every retrieved chunk is relevant — very little noise. This is higher than expected because the |
| Latency = 3.14 s    | CPU only          | 'Time breaks down as: ~0.1 s numpy retrieval, ~3 s Claude API generation. The bottleneck is the LLM     |
| Negation = 1.000    | 8/8 — perfect     | 'A surprise — we expected this to be the hardest category. The system prompt rules worked well for      |
| Factual = 0.792     | 19/24 — weakest   | '5 failures — mostly Kliegman decision-tree terms that fixed-size chunking split at the wrong bound     |

Ablation — this is where you show deep understanding:

| Experiment           | Hit@k | Precision@k | What to say aloud                                                                      |
|----------------------|-------|-------------|----------------------------------------------------------------------------------------|
| fixed k=5 (baseline) | 0.900 | 0.810       | 'Baseline on the first 20 questions.'                                                  |
| paragraph k=5        | 0.900 | 0.870       | 'Same hit rate — but 6% higher precision. Paragraph chunks are more on-target.'        |
| fixed k=3            | 0.850 | 0.850       | 'Precision rises because there is less noise — but we lose one hit on a numerical ques |
| fixed k=8            | 0.900 | 0.787       | 'Hit does not improve beyond k=5 — we only add noise chunks. k=5 is the sweet spot.'   |

Say: Frame the ablation like this: 'I ran 4 experiments to isolate what actually matters. The surprising finding: chunking strategy matters more than  $k$ . Paragraph chunking gives 6% higher precision at zero additional latency cost.  $k=8$  doesn't help because it introduces low-relevance chunks that confuse the model.'

## 6

### Lessons Learned — What I Would Improve

This is the step that separates 'I used a library' from 'I understood the material':

---

**Cross-encoder  
re-ranking**

A bi-encoder is fast but less precise — it encodes the query and chunk independently. A cross-encoder sees both together and is significantly more accurate. I would add ms-marco-MiniLM to re-rank the top-10 results — cost ~0.5 s, worthwhile for temporal and multi-hop questions.

---

**Sentence-boundary  
overlap**

The current 50-char overlap cuts at an arbitrary position mid-sentence. I would compute sentence boundaries first and ensure every chunk starts and ends on a complete sentence. This would improve the 'numerical' category where digits get split across chunks.

---

**LLM-as-Judge  
evaluation**

Manual quality assessment of 10 questions is not enough. I would use Claude Sonnet to verify whether each generated answer is actually supported by its cited passage — a faithfulness check that scales to all 52 questions automatically.

---

**Tip Closing sentence**

Finish with: 'The most important thing I learned is that the technical pipeline is relatively straightforward — the real challenge is diagnosing where the failures are and deciding what to improve first. Ablation analysis is the tool for that.'

# Expected Instructor Questions — and Answers

*Read through this several times before presenting — these are the questions that separate a 90 from a 100.*

## Conceptual Understanding

---

### Q: What is the difference between RAG and fine-tuning?

Fine-tuning forgets — it 'colours' the model weights with new data but can overwrite existing knowledge (catastrophic forgetting). RAG never touches the weights — it injects external context at inference time. RAG is also more up-to-date: you can swap the documents without retraining. Fine-tuning is best for style and fixed tasks; RAG is best for factual knowledge that changes.

### Q: Why a bi-encoder and not a cross-encoder for retrieval?

A cross-encoder is more accurate but  $O(N)$  — it needs a forward pass for every (query, document) pair. For an index of 10 K chunks that means 10,000 model calls per question — roughly 30 s on CPU. A bi-encoder computes the query embedding once (384-dim) and then does a matrix multiply —  $O(1)$  given a prebuilt index. Strategy: bi-encoder for retrieval, cross-encoder for re-ranking if needed.

### Q: Why L2 normalisation?

Without normalisation, the dot product depends on vector magnitude — longer sentences score higher simply because they contain more words. Normalisation makes the metric angle-only (semantic direction) and enables fair comparison between a two-word decision-tree fragment and a long paragraph.

### Q: What is Hit@k versus Precision@k?

Hit@k: binary per question — 1 if at least one correct chunk appears in the top-k, 0 otherwise. Precision@k: averaged fraction — how many of the k retrieved chunks are relevant? Hit@k measures recall (did we find it?). Precision@k measures accuracy (how much noise?). The goal is to maximise Hit while keeping Precision high — k=5 is the balance point.

## The RAG Pipeline

---

### Q: What happens if the correct chunk is not retrieved?

The system answers based only on whatever was retrieved. If no relevant chunk is found, the system prompt tells Claude to say 'The information was not found in the provided sources.' Known failure mode: occasionally the model still cites the closest chunk as a fallback — this was noted in the failure analysis.

### Q: Why chunk size 400 and not 200 or 800?

200: too small — a sentence fragment. The embedding receives little context and retrieval noise grows. 800: too large — two unrelated clinical points merge into one chunk, diluting the semantic signal. 400  $\approx$  80 tokens — about 2 sentences, a coherent context, a distinctive score. Validated by ablation: changing k had little effect, but chunking strategy did.

### Q: What is the difference between dense retrieval and BM25?

BM25 retrieves by keyword matching (a TF-IDF variant) — if the keyword isn't in the text, the chunk won't be returned. Dense retrieval searches by meaning — 'febrile seizure discharge' can surface a page that writes 'criteria for an afebrile child.' Dense is better for natural language; BM25 is better for unique scientific names (drug names).

## Technical Questions

---

### Q: Why did you replace ChromaDB with numpy?

ChromaDB 1.5.9 has an HNSW-specific bug on Windows: the Rust index loads into memory during the build process but is not written to disk correctly. Every new Python process that opened the same path got an `InternalError`. A sleep+verify workaround worked within one process but failed across processes. numpy is sufficient for 10 K vectors, fully portable, and has zero additional dependencies.

**Q: `np.argsort` vs `np.argpartition` — what is the difference?**

`np.argsort` sorts all  $N=10,107$  elements —  $O(N \log N)$ . `np.argpartition` finds the  $k$  largest in  $O(N)$  without a full sort. We then sort only those  $k=5$  elements —  $O(k \log k)$  which is effectively constant. In practice the difference is small at  $N=10\text{ K}$ , but it is the conceptually correct approach.

**Q: Why is query embedding done inside `retrieve()` rather than in `rag_system.py`?**

Separation of concerns: `retrieval.py` owns everything related to the index. `rag_system.py` is a pure orchestrator — it knows nothing about vectors. This also allows `retrieve()` to be tested independently without an LLM.

**Q: What is semantic drift?**

When chunks are too large, the model's attention 'spreads' across the full text and may answer a nearby but different question. Example: a question about otitis media receives an answer about chronic ear disease because both appear in the same oversized chunk.

## What-If Questions

---

**Q: What if the corpus grew to one million documents?**

A numpy flat file is sufficient for  $\sim 100\text{ K}$  vectors in memory. For one million: migrate to FAISS (approximate nearest neighbour, GPU) or Qdrant/Weaviate (distributed). Also shard by medical domain — a question about a sick child doesn't need to search oncology. The retrieval interface doesn't change — only the backend.

**Q: What if a document is updated every day?**

Incremental indexing: each document has a hash. When the hash changes — re-chunk and re-embed only that document, replace the relevant rows in `.npy` and `.json`. At higher scale: a database with upsert-by-ID such as Qdrant.

**Q: What if someone asks a question unrelated to pediatric medicine?**

The system returns the 5 closest chunks from the corpus — a cooking question would receive unrelated medical passages. The system prompt tells Claude to answer 'not found' when the context is not relevant. Improvement: add a domain classifier as a first stage to check whether the question is in scope.

# Quick-Reference Glossary — Terms You Must Get Right

| Term             | Correct definition to say aloud                                                                                                                          | Common mistake to avoid                                                                    |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| RAG              | Injecting externally retrieved context into an LLM prompt, for the LLM to generate an answer to the user's question.                                     | It's just a search-to-retrieve plus specified generation.                                  |
| Embedding        | A vector representation of text in $\mathbb{R}^{384}$ such that semantically similar sentences are close to each other (cosine), not Euclidean distance. | Measuring distances is a measure of distance, not Euclidean distance.                      |
| Chunking         | Splitting a document into pieces sized to enable accurate embedding and retrieval.                                                                       | Splitting into paragraphs is a strategy you choose, not just paragraphs.                   |
| Hit@k            | Binary metric per question: 1 if at least one correct chunk appears in the top k retrieved chunks.                                                       | Don't confuse with Precision. Otherwise measures recall, not accuracy.                     |
| Precision@k      | What fraction of the k retrieved chunks are relevant? Average over all questions.                                                                        | Not about accuracy — it measures the chunks, not the questions.                            |
| Bi-encoder       | A model that encodes query and document independently into vectors.                                                                                      | Don't confuse with cross-encoder which sees query + document together.                     |
| L2 Normalisation | Dividing a vector by its norm so $\ v\ =1$ . After this, cosine similarity is dot product.                                                               | Don't say it's dot product without explaining why — the answer: so we can use dot product. |
| Ablation Study   | An experiment that changes one parameter at a time to isolate the effect of that parameter.                                                              | Not a test or a performance systematic experiment, not debugging.                          |

## Do and Don't During the Presentation

| DO                                                                                                                                                    | DON'T                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Talk about the ChromaDB bug — it shows you hit a real problem                                                                                         | Say 'everything worked smoothly' — an instructor won't believe it   |
| Frame the ablation as a systematic experiment with a hypothesis                                                                                       | Say 'I just tried a few values'                                     |
| Show real code in the terminal — not only slides                                                                                                      | Rely solely on the PDF — this is a code demonstration               |
| Say 'I didn't know' and then explain what you learned                                                                                                 | Invent an answer to something you don't know — instructors notice   |
| Name the tradeoff in every decision (why k=5 and not k=3)                                                                                             | Say 'I chose X because X works' — there is always something to lose |
| Run one live query: <code>python -c "from src.rag_system import answer; print(answer('What are the discharge criteria for febrile seizure?'))"</code> | Skip the live demo — it is the strongest moment of the presentation |

## Live Demo Script — Run This Before the Presentation

Command to run in the terminal from the project directory:

```
# Run a single question
python -c "from src.rag_system import answer; import json; r = answer('What are the discharge criteria for febrile seizure?'); print(r['answer']); print('Sources:', r['sources'])"
```

Run the full evaluation (results saved to `eval_run.log`):

```
python eval/run_eval.py --strategy fixed --k 5
```

Run the ablation study:

```
python eval/run_eval.py --ablation
```

Tip Practice exercise

Run one question live and explain every output line: what `chunk_id` means, what the score represents, and why the answer cites exactly those sources. If there is an error — that is fine, say: 'Interesting error, let's understand it together.'

Good luck! — The project is solid, the numbers are real, and the code works. All that remains is to explain it with confidence.